

Пояснительная записка

Веб-приложение для проведения интерактивных опросов «QuizLive»

1. Цель и описание проекта

Разработан MVP веб-приложения для проведения интерактивных квизов в реальном времени. Платформа позволяет организаторам создавать викторины и запускать их на мероприятиях, а участникам — подключаться по коду комнаты и отвечать на вопросы в режиме реального времени.

Реализованная функциональность:

- Регистрация и авторизация пользователей двух ролей: **участник** и **организатор**
 - Создание и настройка квизов: выбор категории, настройка времени на вопрос
 - Добавление вопросов двух типов: с одиночным и множественным выбором ответа
 - Поддержка вопросов с изображениями (загрузка файла организатором)
 - Запуск квиза с генерацией 6-значного кода комнаты
 - Отображение вопросов в реальном времени для всех участников через WebSocket
 - Серверный таймер и автоматический переход к следующему вопросу
 - Система начисления очков с бонусом за скорость ответа
 - Лидерборд в реальном времени после каждого вопроса
 - Личный кабинет с историей участия/проведения квизов
-

2. Обоснование выбора средств разработки

2.1 Backend: Node.js + Express + Socket.IO

Выбор Node.js обусловлен единым языком разработки на всём стеке (JavaScript), что ускоряет разработку и упрощает поддержку кода. Express — минималистичный и широко распространённый фреймворк для построения REST API, не навязывающий лишних абстракций для MVP.

Socket.IO выбран для real-time взаимодействия как наиболее зрелая и стабильная реализация WebSocket с автоматическим fallback на long-polling, встроенным механизмом комнат (rooms) и надёжной обработкой переподключений — что критично для живых квизов на мероприятиях.

2.2 База данных: SQLite (better-sqlite3)

Для MVP выбрана встроенная база данных SQLite: не требует отдельного сервера, развёртывается в одну команду, обеспечивает полноценный SQL и достаточную производительность для мероприятий до ~500 участников. Библиотека `better-sqlite3` обеспечивает синхронный API, что упрощает код и устраняет типичные ошибки с `async/await` при работе с БД в Node.js.

2.3 Аутентификация: JWT

JSON Web Tokens обеспечивают stateless-аутентификацию без необходимости хранить сессии на сервере. Токен передаётся как в HTTP-заголовках (REST API), так и в параметрах Socket.IO событий, что позволяет использовать единый механизм авторизации для обоих протоколов.

2.4 Frontend: React + Vite + React Router

React выбран как наиболее популярный фреймворк с развитой экосистемой, отлично подходящий для построения динамичного UI с частыми обновлениями состояния (таймер, лидерборд, список участников обновляются в реальном времени).

Vite обеспечивает мгновенный hot-reload при разработке и быструю сборку за счёт использования нативных ES-модулей браузера.

React Router v6 — стандарт маршрутизации для React SPA с удобным API вложенных маршрутов.

Для стилизации использован собственный CSS без UI-библиотек: это исключает зависимость от сторонних компонентов, уменьшает bundle и позволяет полностью контролировать внешний вид интерфейса.

3. Архитектура системы



Схема базы данных включает 7 таблиц:

- **users** — пользователи (роль, email, хэш пароля)
- **quizzes** — квизы организаторов
- **questions** — вопросы квиза (текст, тип, изображение)
- **options** — варианты ответов к вопросам
- **sessions** — запущенные квиз-сессии (код комнаты, статус)
- **session_participants** — участники сессии
- **participant_answers** — ответы участников (для подсчёта очков)

4. Основные этапы разработки

Этап 1 — Проектирование (моделирование БД и UI)

Спроектирована реляционная схема данных с учётом требований: хранение ответов участников, поддержка нескольких правильных вариантов, история сессий. Созданы макеты основных экранов в Figma.

Этап 2 — Backend: REST API

Реализованы маршруты авторизации, CRUD для квизов и вопросов (включая загрузку изображений через `multer`), управление сессиями. Настроена JWT-аутентификация с middleware для проверки ролей.

Этап 3 — Real-time ядро (Socket.IO)

Реализован цикл квиза: лобби → обратный отсчёт → вопросы → результаты → лидерборд. Серверный таймер (`setTimeout`) автоматически закрывает приём ответов и переходит к следующему вопросу. Реализована система комнат по коду (Socket.IO rooms).

Этап 4 — Система очков

Для одиночного выбора: $\text{score} = \text{round}(500 + 500 \times (1 - t/T))$, где `t` — время ответа, `T` — лимит. При верном ответе: 500–1000 очков в зависимости от скорости. Для множественного выбора: пропорциональный расчёт с учётом доли правильно выбранных вариантов и штрафа за неверные.

Этап 5 — Frontend

Разработан адаптивный интерфейс: страницы авторизации, дашборд организатора с управлением квизами, редактор вопросов, экран живого квиза (таймер, варианты ответов, лидерборд), интерфейс участника. Реализованы анимации обратной связи (результат ответа, появление участников).

Этап 6 — Интеграция и тестирование

Проверен полный сценарий: регистрация → создание квиза → добавление вопросов → запуск сессии → подключение участников → проведение квиза → итоговый лидерборд.

5. Заключение

Разработан работоспособный MVP платформы для интерактивных квизов, реализующий все обязательные требования задания. Архитектурные решения обеспечивают возможность масштабирования: замена SQLite на PostgreSQL, подключение Redis для масштабирования

Socket.IO на несколько серверов, и развёртывание на облачных платформах (Railway, Render, Vercel).

Приложения

	Ссылка
Макеты (Figma)	<i>вставить ссылку</i>
Репозиторий (GitHub)	<i>вставить ссылку</i>
Работоспособный продукт	— (запускается локально, см. README)